Anchored SubgraphRAG: Improving Graph-RAG with Structural Awareness

Konstantin Sturtzkopf
University of Cambridge
kcs41@cam.ac.uk

Abstract

SubgraphRAG scores knowledge-graph triples independently, losing structural coherence on multi-hop questions. Its ablations show that adding a GNN degrades retrieval performance due to semantic diffusion noise. We propose Anchored SubgraphRAG (ASR), which reverses this finding: by gating GNN message passing with purely structural signals, we show that local graph context *improves* triple scoring. ASR selects semantic anchor triples, expands a local subgraph via BFS, and refines entity representations before scoring. On WebQSP, ASR achieves triple recall@100 of 0.905 vs. SubgraphRAG's 0.883, with the largest gains on multi-hop queries. Ablations confirm that the GNN is the largest contributor ($+5.4$pp), but only when structurally gated.

GitHub repository

The companion source code for this project can be found at: https://github.com/ksturtzkopf/cambridge-l65.

1 Introduction

Large Language Models (LLMs) show remarkable capacity for question answering when provided with the right context, but tend to hallucinate when that context is missing or incomplete [3]. *Retrieval-augmented generation* (RAG) addresses this by grounding LLM responses in external knowledge added to the context before answering the question [7]. Knowledge graphs are a particularly well-structured source [4]. SubgraphRAG represents the current state of the art in KG-based RAG, using a lightweight MLP to score individual triples from a graph for a given question, and passing the top-k triples to an LLM for reasoning [4].

However, SubgraphRAG's retriever scores triples independently of one another. For single-hop questions, where the answer entity is directly connected to the topic entity, this works well. For multi-hop questions, which require reasoning over a chain of intermediate triples, structurally coherent paths can be lost in favour of individually high-scoring but disconnected triples. SubgraphRAG's own evaluation confirms this: performance degrades consistently as hop count increases, and the authors note that addressing particularly challenging multi-hop queries remains beyond their current scope.

We introduce *Anchored SubgraphRAG* (ASR), a structure-aware extension to SubgraphRAG's pipeline, inspired by the anchor-based structural context of Alsentzer et al. [1]. ASR selects anchor triples by semantic similarity to the question, constructs a local subgraph from the topic entity via BFS, and refines entity representations through a positional-encoding-gated GNN before scoring. Critically, message passing is controlled and gated entirely by structural signals, avoiding diffusion noise that caused earlier GNN-based retrievers to underperform [4]. We evaluate the architecture on WebQSP, analysing failure modes on different numbers of hops, and show preliminary results of this architecture outperforming the SubgraphRAG retrieval results.

Konstantin Sturtzkopf, Anchored SubgraphRAG: Improving Graph-RAG with Structural Awareness. *Geometric Deep Learning (L65), University of Cambridge*.

2 Background

Knowledge-Graph Retrieval Augmented Generation (KG-RAG) KG-RAG retrieves triples from a knowledge graph $\mathcal{G} = \{(h, r, t) \mid h, t \in \mathcal{V} \wedge r \in \mathcal{R}\}$ to ground LLM reasoning [4, 6], with $\mathcal{V}$ being the set of nodes and $\mathcal{R}$ the set of relationships.

These systems face a fundamental trade-off between retrieval expressiveness and computational cost. Retrievers should remain lightweight and flexible, delegating reasoning complexity to downstream models as Li et al. [4] argue.

They introduce *SubgraphRAG*, which uses a retrieve-then-reason paradigm. They retrieve a candidate subgraph for a given natural language query using a lightweight model and use a natural language version of this subgraph for reasoning in a more powerful LLM afterward. The retrieval module works by identifying the *topic entities* of a query first. A topic entity is the entity explicitly mentioned in the question, serving as a retrieval anchor point [4]. For the k-hop neighbourhood of the topic entities, SubgraphRAG calculates a directional distance encoding (DDE) which roughly measures how far away each node is from a question's topic entities. After, each triple within the graph is scored independently with DDE as context by a lightweight MLP. Li et al. [4] identify diffusion of semantic information in GNN-based aggregators as a failure mode, as this may introduce noise in the triple selection. The authors found that scoring each triple using only its own embeddings and distance encoding performs best, but their retriever, critically, does not incorporate local graph context around each entity before scoring.

Subgraph Classification Outside KG-RAG, Alsentzer et al. [1] show that scoring subgraphs with structural context from sampled anchor patches outperforms scoring in isolation.

To the best of our knowledge, no existing work applies this principle effectively to knowledge graph retrieval.

3 Architecture

Given a question in natural language q and a knowledge-graph $\mathcal{G}$, the task of the model is to rank candidate triples, such that the triples $(h, r, t) \in \mathcal{G}$ relevant to answering q receive the highest score.

Figure 1 shows our five-stage architecture: anchor selection, subgraph expansion, positional encoding, GNN encoding, as well as triple scoring.

Anchor Selection We first select *anchor triples*. We embed all triples as the mean of the component embeddings and calculate the cosine similarity of the triple and query embeddings, to select the top-k triples as anchors. These anchors are, by construction, the triples that are semantically most similar to the question.

Subgraph Expansion Given the top-k anchor triples and topic entities, we construct a local subgraph in three steps. We form a seed set from all topic entities and the endpoints of anchor triples. We expand this set by H-hop BFS (breadth-first-search) over the (undirected) candidate graph, and keep every triple (h, r, t) whose head and tail both lie in the expanded set. Only entities in this subgraph are refined by the following GNN; all others keep their original embeddings.

Positional Encoding For each node $v \in \mathcal{V}$, we calculate a positional encoding $\mathbf{p}_v$ based on three structural signals. First, we calculate *topic DDE*, adopted from Li et al. [4]: starting from the one-hot indicator whether a node is a topic entity, we run T rounds of mean-aggregation along directed edges of the graph, as well as T rounds along reversed edges. We concatenate the intermediate states of each entity's directional distance from the topics.

Next, we calculate an *anchor proximity encoding* as the BFS distance (clamped to a hyperparameter $\mathrm{d_clamp}$) on the undirected graph from v to each anchor entity. The distance is embedded via a learned lookup table and averaged across anchors.

Lastly, we create *path indicators*. Path indicators are seven binary features recording whether the entity lies on a shortest path between topic and anchor, or is a direct neighbour of either. We also look up the degree of the node v and create a one-hot degree bin depending on the number of nodes v is directly connected to.

GNN We refine entity representations of our expanded subgraph with a stack of $\mathcal{L}$ relational gated convolution layers. The relational gating is inspired by Li et al. [5]. For every directed edge (h, r, t)

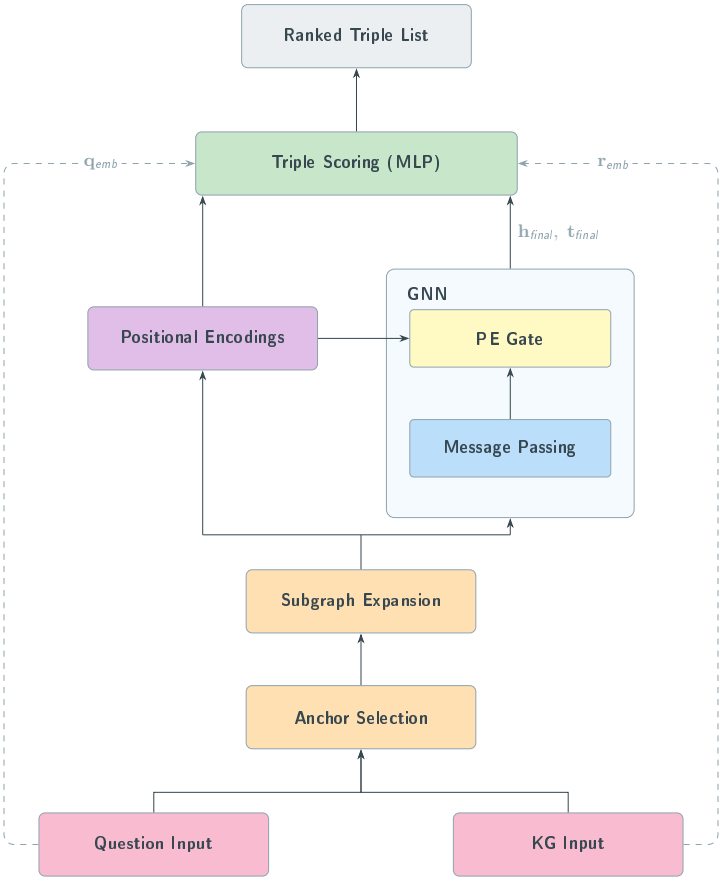

Figure 1: Architecture of Anchored SubgraphRAG

in our KG and layer $1 \leq l+1 \leq \mathcal{L}$ we calculate

$$m_{h\to t}^{(l+1)} = \mathrm{ReLU}\Big(\mathbf{W}_{\mathrm{msg}}[\mathbf{x}_h^{(l)} \,\|\, \mathbf{e}_r]\Big)\,, \tag{1}$$

where $\mathbf{x}_h$ is the current representation of entity h, and $\mathbf{e}_r$ is a learned projection of the relation.

Before aggregation, we also scale each message by a scalar gate

$$g_{h\to t} = \sigma\big(\mathrm{MLP}([\mathbf{p}_h \,\|\, \mathbf{p}_t])\big), \tag{2}$$

where $\mathbf{p}_h, \mathbf{p}_t$ are the positional encodings, only encoding structural information calculated for the nodes in the previous step. As such, the gate **receives no semantic input**.

For each layer $1 \leq l+1 \leq \mathcal{L}$ in the GNN-Stack, we update each node $\mathbf{t}$

$$\mathbf{x}_t^{(l+1)} = \mathrm{ReLU}(\mathbf{W}_{\mathrm{upd}}\left[\frac{1}{|\mathcal{N}(t)|}\sum_{(h,r,t)\in\mathcal{N}(t)}\left(g_{h\to t}\cdot m_{h\to t}^{(l+1)}\right) \,\|\, \mathbf{x}_t^{(l)}\right]), \tag{3}$$

where $\mathcal{N}(t)$ is the neighbourhood of node t and $\mathbf{x}_t^0 = \mathbf{t}_{\mathrm{emb}}$. For each node v, the final embedding we pass to the following layers is $v_{\mathrm{final}} = \mathbf{x}_v^{\mathcal{L}}$. We refer to this final embedding as the *contextualised embedding* of v.

Triple Scoring For each triple (h, r, t) and a given question q, our triple scorer is a simple MLP that, based on the results of the previous layers, calculates

$$\mathrm{score}(q,(h,r,t)) = \mathrm{MLP}([\mathbf{q}_{\mathrm{emb}} \,\|\, \mathbf{h}_{\mathrm{final}} \,\|\, \mathbf{r}_{\mathrm{emb}} \,\|\, \mathbf{t}_{\mathrm{final}} \,\|\, \mathbf{p}_h \,\|\, \mathbf{p}_t]),$$

where q_{emb} is the embedding of the question, $\mathbf{h}_{\mathrm{final}}, \mathbf{t}_{\mathrm{final}}$ are the contextualised embeddings for h and t respectively, and $\mathbf{r}_{\mathrm{emb}}$ is the embedding of the relation between h, t. $\mathbf{p}_h, \mathbf{p}_t$ are the positional encodings of h, t respectively.

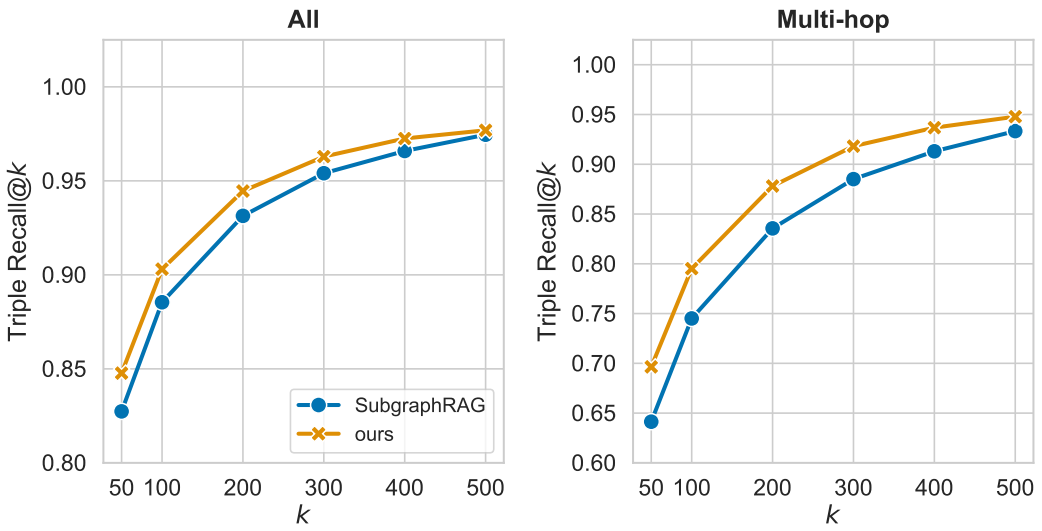

Figure 2: `Triple recall@k` on the test set, split by overall and multi-hop performance for the WebQSP. Based on one run of SubgraphRAG and our method each.

The resulting architecture has 5.3M parameters, with the Triple Scoring being responsible for the majority of them (4.2M). This is an increase of one million parameters compared to the SubgraphRAG implementation, which is mostly due to the added GNN.

4 Training and Evaluation

Akin to Li et al. [4], training is based on weak supervision derived from the graph structure: triples on any shortest path between a question's topic entities and its known answer entities receive label 1. We train the model using binary cross-entropy to rank these triples above the remaining ones.

We evaluate and compare models based on `triple recall@k`, where k is the number of triples being retrieved. The metric measures the fraction of ground-truth, shortest-path triples that appear among the k triples with the highest score given by the retriever. This is consistent with the evaluation strategy of Li et al. [4]. We do not evaluate the reasoning result that LLMs achieve with the retrieved triples. We also report the `entity recall@100`, which measures how often the ground truth answer entity is present in the retrieved triples.

Figure 2 shows `triple recall@k` as a function of the number of retrieved triples on the test set. Notably, our architecture outperforms the SubgraphRAG baseline implementation for every value of k displayed. Li et al. [4] outlined that multi-hop questions are particularly challenging for their model. Here, we can see the largest difference in performance between our model and SubgraphRAG. For $k = 100$, we can observe that our model performs approximately 5 percentage points better than SubgraphRAG in multi-hop questions.

We run our model three times to verify robustness on three different seeds, and compare the averaged `triple recall@100` to the results Li et al. [4] discuss in their paper. Our method achieves an average *overall* `triple recall@100` of 0.905 ± 0.0095, whereas Li et al. [4] report a score of 0.883 for their model. This represents an approximately 2.2 percentage point increase.

We run ablations for our model to verify that the individual components of the architecture are required. Each ablation is run once due to a lack of compute for multiple runs. Table 1 shows the results. The GNN contributes the largest single improvement, and we interpret these results in Section 5.

	TR@50	TR@100	TR@200	ER@100
SubgraphRAG	82.7 (-2.0)	88.5 (-2.0)	93.1 (-1.3)	91.5 (-2.3)
Ours	**84.7**	**90.5**	**94.4**	**93.8**
w/o Topic PE	83.6 (-1.1)	89.8 (-0.7)	94.3 (-0.1)	92.9 (-0.9)
w/o Anchor PE	84.1 (-0.6)	89.8 (-0.7)	94.2 (-0.2)	93.0 (-0.8)
w/o Path PE	83.3 (-1.4)	89.3 (-1.2)	93.9 (-0.5)	92.7 (-1.1)
w/o All PE	81.6 (-3.1)	87.7 (-2.8)	92.4 (-2.0)	92.1 (-1.7)
w/o Gating	83.2 (-1.5)	89.5 (-1.0)	93.8 (-0.6)	93.1 (-0.7)
w/o GNN	78.3 (-6.4)	85.1 (-5.4)	90.1 (-4.3)	92.1 (-1.7)

Table 1: Ablation results on WebQSP test set (all questions), deltas shown relative to our full model. TR: Triple Recall, ER: Entity Recall. All configurations run once; *Ours* and *w/o Gating* show averages over three training runs with different seeds. Results are rounded before deltas are calculated.

5 Discussion & Limitations

SubgraphRAG's ablations showed that adding a GNN *hurts* retrieval and suggest this is due to semantic diffusion noise [4]. Our results reverse this finding, and Table 1 shows that the largest single contributor to improved recall is the GNN ($+5.4$pp). However, the mechanism matters: structural gating is critical to realising this gain. Removing structural gating while keeping the GNN (*w/o Gating*) halves the gain over SubgraphRAG at recall@100 (from 2pp to 1pp), confirming that the GNN's value depends on restricting message passing to structural signals. Removing all positional encodings collapses performance below the SubgraphRAG baseline, confirming that the GNN's value comes from propagating *structural* context, not semantic features.

We hypothesise that structural gating preserves semantic diversity in the contextualised embeddings because it amplifies or suppresses messages based on *where* a neighbour sits in the graph, not *what* it represents. A semantic gate would correlate with message content, amplifying already-similar neighbours and creating a feedback loop that accelerates oversmoothing. A structural gate is independent of the semantic embeddings being passed, allowing positionally relevant updates through without biasing the aggregation towards a neighbourhood mean.

Anchors are selected by semantic similarity to the question, and they naturally neighbour answer entities, causing path indicators to correlate with training labels. As the anchor selection is dependent only on the question, not the answer entity, this reflects effective anchor placement, not label leakage. The path indicator ablation (-1.2pp for triple-recall@100, Table 1) confirms these features contribute to but do not dominate performance.

Unlike Alsentzer et al. [1], who sample anchor patches randomly for subgraph classification, our anchors are query-dependent: selected by semantic similarity to the question, they provide task-specific positional reference points that shift with each query. This generates the structural context ad-hoc rather than fixed, which is essential for a retrieval setting where relevance is defined per-question.

The largest gains appear on multi-hop questions, but multi-hop recall still lags behind single-hop. We hypothesise this is partly due to the limited number of GNN layers, which restricts how far structural context can propagate through the subgraph.

Additionally, our weak supervision setup assigns negative labels to relevant triples not on shortest paths, which may impoverish the multi-hop training signal. Future work should explore query-aware labelling strategies to address this. It should also evaluate ASR on more complex datasets such as CWQ, explore hyperparameter sensitivity, and test whether other GNN architectures, such as a gated version of GraphSAGE [2], can further improve performance.

We have evaluated retrieval in isolation, and whether the improved triple recall translates to downstream QA gains remains to be verified.

6 Conclusion

We show that the tension between GNNs and retrieval quality in KG-RAG can be resolved by restricting message passing with structural signals. Anchored SubgraphRAG demonstrates that local graph context improves triple scoring when semantic diffusion is controlled, with preliminary results showing a 2.2pp improvement over SubgraphRAG on WebQSP with the largest gains on multi-hop queries where structural coherence matters most.

Acknowledgements

We want to thank Lorenzo Giusti for his invaluable support, availability for meetings, great discussions, and for being an incredibly helpful advisor.

References

[1] Emily Alsentzer et al. *Subgraph Neural Networks*. arXiv:2006.10538 [cs]. Nov. 2020. DOI: 10.48550/arXiv.2006.10538. URL: http://arxiv.org/abs/2006.10538 (visited on 02/25/2026). 1, 2, 5

[2] William L. Hamilton et al. "Inductive Representation Learning on Large Graphs". In: *NIPS*. 2017, pp. 1024–1034. 5

[3] Ziwei Ji et al. "Survey of Hallucination in Natural Language Generation". In: *ACM Computing Surveys* 55.12 (Dec. 2023). arXiv:2202.03629 [cs], pp. 1–38. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3571730. URL: http://arxiv.org/abs/2202.03629 (visited on 02/25/2026). 1

[4] Mufei Li et al. *Simple Is Effective: The Roles of Graphs and Large Language Models in Knowledge-Graph-Based Retrieval-Augmented Generation*. arXiv:2410.20724 [cs]. Feb. 2025. DOI: 10.48550/arXiv.2410.20724. URL: http://arxiv.org/abs/2410.20724 (visited on 02/25/2026). 1, 2, 4, 5

[5] Yujia Li et al. "Gated Graph Sequence Neural Networks". In: *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*. Ed. by Yoshua Bengio et al. 2016. URL: http://arxiv.org/abs/1511.05493. 2

[6] Shirui Pan et al. "Unifying Large Language Models and Knowledge Graphs: A Roadmap". In: *IEEE Transactions on Knowledge and Data Engineering* 36.7 (July 2024). arXiv:2306.08302 [cs], pp. 3580–3599. ISSN: 1041-4347, 1558-2191, 2326-3865. DOI: 10.1109/TKDE.2024.3352100. URL: http://arxiv.org/abs/2306.08302 (visited on 02/27/2026). 2

[7] Kurt Shuster et al. *Retrieval Augmentation Reduces Hallucination in Conversation*. arXiv:2104.07567 [cs]. Apr. 2021. DOI: 10.48550/arXiv.2104.07567. URL: http://arxiv.org/abs/2104.07567 (visited on 02/25/2026). 1

A Appendix

A.1 Configuration

Default seed is 42. When stated that models were run on multiple seeds, they were run on seeds 2002 and 1209 as well.

Parameter	Value	Description
emb_dim	1024	Dimensionality of the pre-trained node/entity embeddings fed into the model.
hidden_dim	256	Size of the internal GNN hidden representations.
rel_dim	128	Dimensionality of relation-type embeddings.
pe_anchor_dim	16	Dimensionality of the anchor-based positional encoding features.
pe_max_dist	5	Maximum shortest-path distance considered when computing anchor positional encodings (distances beyond this are clipped).
num_anchors	64	Number of anchor nodes sampled for anchor-based positional encoding.
subgraph_hops	1	Number of hops used to extract the local subgraph around a target link.
gnn_layers	2	Number of message-passing layers in the GNN encoder.
dde_rounds	2	Number of forward Double-Radius Distance Encoding rounds for topic-aware positional features.
dde_reverse_rounds	2	Number of reverse DDE rounds, capturing distance information from the opposite direction.
scorer_dropout	0.1	Dropout rate applied in the final link-scoring MLP head.
gnn_dropout	0.1	Dropout rate applied within the GNN message-passing layers.
lr	5e-4	Learning rate for the optimiser.
weight_decay	1e-2	L2 regularization coefficient to prevent overfitting.
patience	10	Number of validation epochs without improvement before early stopping triggers.

Table 2: Default hyperparameter configuration for ASR